>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 14

Basic File Operations · Semester Review

text files · paths · File and Stream classes · CSV · exceptions · review of Weeks 1–14 · mini-project

```
> dotnet run
Created proverbs.txt
…
Lines     : 5
Words     : 15
Characters: 98
Letters   : 83
```

Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the end of this week you can …

Files and paths

Explain why files are needed; know where a relative path points (working directory).

The File class

WriteAllText, AppendAllText, ReadAllText, WriteAllLines / ReadAllLines with arrays.

Streams

StreamWriter and StreamReader inside using; read with ReadLine() until null.

CSV data

Split(',') a line into fields, int.Parse them, compute row totals, write a report.

Exceptions

try / catch for FileNotFoundException, IOException and FormatException.

Review + project

Connect all topics of the semester in the Student Grade Manager.

So far, our data was written into the code

Week 13 · data in the code

```
string[] names = { "Ayse", "Mehmet", "Can" };
// data inside Program.cs:
// change it = edit + compile again
```

Week 14 · data in a file

```
string[] lines = File.ReadAllLines("grades.csv");
// data in a file next to the program:
// edit it, save it, run again
```

return
methods hand back values: LoadTasks returns the lines of a file

arrays as parameters
one method fills the arrays, another one prints them

scope and lifetime
locals die when the method ends — only a file survives the program

Today: data outlives the program. Save it to a text file, load it at the next start.

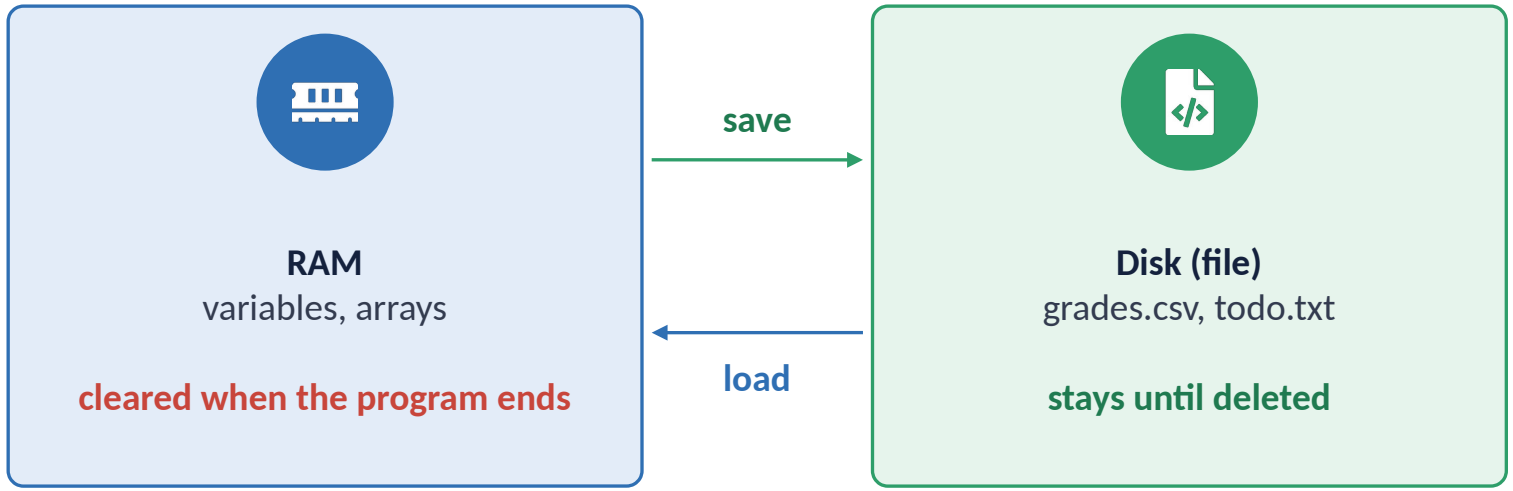


PART 1

Text files

Paths · the File class · lines ⇄ arrays · StreamWriter / StreamReader · CSV · exceptions

RAM forgets, the disk remembers



	Text file	Binary file
Contains	characters (UTF-8)	raw bytes
Open with	Notepad, VS Code	its own program
Examples	.txt .csv .log .cs	.exe .dll .png .docx
This course	✓ Weeks 14–15	—

Persistence
to-do lists, high scores, balances survive the next start

Exchange
CSV files open in Excel and in other programs

Logs
a log file shows later what happened and when

A relative path starts in the working directory

```
14_01_FirstFile/                ← working directory
├── FirstFile.csproj
├── Program.cs
├── notes.txt                   ← "notes.txt"
├── backup/
│   └── notes_copy.txt          ← Path.Combine(…)
└── bin/Debug/net10.0/          ← BaseDirectory (.dll)
```

14_01_FirstFile · real output

```
Working directory: 14_01_FirstFile
Program files in : bin/Debug/net10.0/
Copy saved as backup/notes_copy.txt
```

Path	Kind
`"notes.txt"`	relative ✓
`Path.Combine("backup", "a.txt")`	relative ✓
`@"C:\Users\ali\notes.txt"`	absolute ✗



dotnet run inside the project folder → files appear next to Program.cs.



Windows: Path.Combine writes backup\notes_copy.txt — same file, other separator.

One call to write, append or read a whole file

Method	What it does	Gives back
`File.WriteAllText(path, text)`	creates the file or REPLACES its content	—
`File.AppendAllText(path, text)`	adds text at the end (creates if missing)	—
`File.ReadAllText(path)`	reads the whole file	`string`
`File.WriteAllLines(path, lines)`	one array element per line	—
`File.ReadAllLines(path)`	one line per array element	`string[]`
`File.AppendAllLines(path, lines)`	adds lines at the end	—
`File.Exists(path)`	is the file there?	`bool`
`File.Delete(path)`	removes it (no error if missing)	—
`File.Copy(from, to, true)`	copies; true = overwrite the target	—

Rule of thumb: **Write = start again · Append = add · Read = load**

>_ EXAMPLE 1 · FIRST FILE

Write, append, read — and overwrite by mistake

14_01_FirstFile · steps 2–4

```
string path = "notes.txt";
File.WriteAllText(path, "MCMP113 notes\nWeek 14: files\n");
Console.WriteLine($"Created? {File.Exists(path)}");

File.AppendAllText(path, "Data outlives the program.\n");

string content = File.ReadAllText(path);
Console.WriteLine($"{path}: {content.Length} characters");
Console.Write(content);
```

step 6 · replaces the content

```
File.WriteAllText(path, "Only this line is left.\n");
Console.Write($"Now: {File.ReadAllText(path)}");
```

Console

```
Created? True
notes.txt: 56 characters
MCMP113 notes
Week 14: files
Data outlives the program.
---
Copy saved as backup/notes_copy.txt
Now: Only this line is left.
Deleted? True
Copy kept? True
```

WriteAllLines / ReadAllLines: one element = one line

save: double[] → string[] → file

```
string[] lines = new string[week.Length];
for (int i = 0; i < week.Length; i++)
{
    lines[i] = week[i].ToString();
}
File.WriteAllLines(path, lines);
```

append one line · load

```
File.AppendAllLines(path, new string[] { "21.0" });
string[] loaded = File.ReadAllLines(path);
```

temperatures.txt

```
14.5
16
13.5
18
19.5
17
15.5
21.0
```

loaded : string[8]

"14.5"	"16"	"13.5"	"…"
0	1	2	3

ReadAllLines → **loaded[1]** is "16"; **loaded.Length** is 8

Console

```
Average: 16.88 C
Warmest: day 8 with 21.0 C
```

StreamWriter and StreamReader: line by line, inside using

write: file stays open in the loop

```
using (StreamWriter writer = new StreamWriter(path))
{
    writer.WriteLine($"Multiplication table for {n}");
    for (int i = 1; i <= 10; i++)
    {
        writer.WriteLine($"{n} x {i,2} = {n * i,3}");
    }
}
```

read: until ReadLine() returns null

```
string line = reader.ReadLine();
while (line != null)
{
    lineNo++;
    Console.WriteLine($"{lineNo,2}| {line}");
    line = reader.ReadLine();
}
```

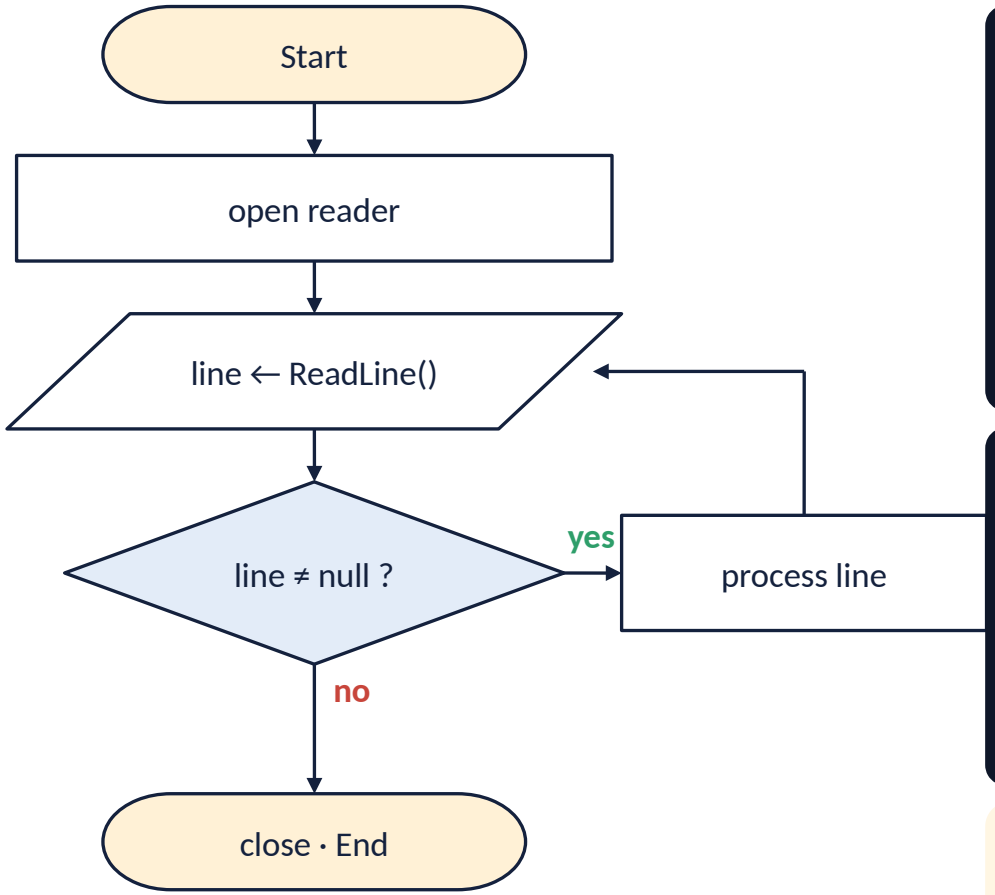
Console

```
Multiplication table for n = 7
Saved table_7.txt
 1| Multiplication table for 7
 2| 7 x  1 =   7
…
```

Console

```
11| 7 x 10 =  70
11 lines read
Now 12 lines, the last one is: -- end of table --
```

ReadLine() until null — as a flowchart and two C# forms



Form A (Example 3) · read before and at the end

```
string line = reader.ReadLine();
while (line != null)
{
    // process line
    line = reader.ReadLine();
}
```

Form B (Example 9) · read and test together

```
string line;
while ((line = reader.ReadLine()) != null)
{
    // process line
}
```

Bug: two ReadLine() calls per pass skip every 2nd line.

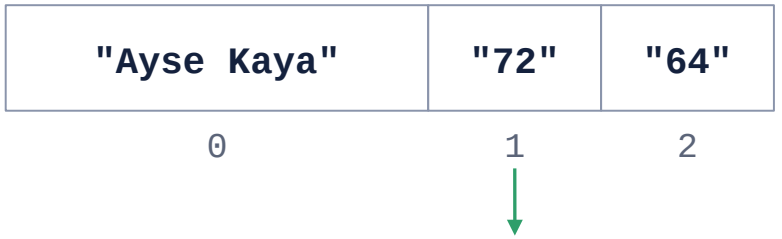
Comma-separated values: one record per line

grades.csv

```
name,midterm,final
Ayse Kaya,72,64
Burak Demir,55,66
Ceren Yildiz,88,95
Deniz Arslan,58,61
Emre Sahin,30,41
```

`lines[1].Split(',')`

`"Ayse Kaya,72,64"`



`fields` string[3]

`int.Parse` → 72 and 64 (numbers)

0.4 · 72 + 0.6 · 64 = **67.2**

Week 9: fields is a string[]; fields.Length is 3.

Header row
start the loop at i = 1

No commas in values
a comma would create an extra field

Decimal point
write 72.5, not 72,5

Parse each row, compute, write a report file

14_04_GradebookCsv · the loop over the rows (shortened: sum and pass count omitted)

```
for (int i = 1; i < lines.Length; i++)              // i = 1: skip the header row
{
    string[] fields = lines[i].Split(',');          // "Ayse Kaya,72,64" -> 3 fields
    string name = fields[0];
    int midterm = int.Parse(fields[1]);
    int final = int.Parse(fields[2]);
    double average = 0.4 * midterm + 0.6 * final;   // MCMP113 weights: 40 % + 60 %
    string letter = LetterGrade(average);
    report.WriteLine($"{name,-14}{midterm,4}{final,5}{average,7:F1}  {letter}");
}
```

report.txt (printed with File.ReadAllText)

```
Student        Mid  Fin    Avg  Grade
Ayse Kaya       72   64   67.2  DC
Burak Demir     55   66   61.6  DD
Ceren Yildiz    88   95   92.2  AA
Deniz Arslan    58   61   59.8  FF
Emre Sahin      30   41   36.6  FF
Class average: 63.48, passed: 3 of 5
```

Why i = 1? line 0 is the header; `int.Parse("midterm")` throws a FormatException.

A to-do list that is still there tomorrow

add = append one line

```
Console.Write("New task: ");
string task = Console.ReadLine();
File.AppendAllText(path, task + Environment.NewLine);
Console.WriteLine("Saved.");
```

load (first run: empty array)

```
static string[] LoadTasks(string path)
{
    if (!File.Exists(path))
    {
        return new string[0];
    }
    return File.ReadAllLines(path);
}
```

run 1 · end

```
  1. Revise arrays for the final
  2. Finish lab 14 exercises
  3. Email the project group
1 List  2 Add  3 Done  0 Quit > 0
Bye! Your list is saved.
```

run 2 · the list was loaded

```
Welcome back! 3 task(s) loaded from todo.txt.
1 List  2 Add  3 Done  0 Quit > 1
  1. Revise arrays for the final
  2. Finish lab 14 exercises
  3. Email the project group
Number of the finished task: 2
```

>_ EXAMPLE 6 · LOG FILE

Append one timestamped line per event

14_06_LogFile · one line per event (called for every reading)

```
// Appends one line with the current date and time to the log file
static void Log(string path, string level, string message)
{
    string line = $"{DateTime.Now:yyyy-MM-dd HH:mm} | {level} | {message}";
    File.AppendAllText(path, line + Environment.NewLine);
}
```

sensor.log after run 2 (printed by the program)

```
--- sensor.log: 8 lines ---
2026-10-12 21:25 | INFO | Program started
2026-10-12 21:25 | INFO | Reading 1: 22.5 C
2026-10-12 21:25 | WARN | Reading 2: 31.2 C is too high
2026-10-12 21:25 | INFO | Reading 3: 27.0 C
2026-10-12 21:25 | INFO | Program finished
2026-10-12 21:25 | INFO | Program started
2026-10-12 21:25 | INFO | Reading 1: 19.5 C
2026-10-12 21:25 | INFO | Program finished
```

>_ EXAMPLE 7 · COUNTING

Lines, words, characters — and Turkish letters

one pass over the lines

```
for (int i = 0; i < lines.Length; i++)
{
    words += CountWords(lines[i]);
    characters += lines[i].Length;
    letters += CountLetters(lines[i]);
    // ... remember the longest line
}
```

proverbs.txt (UTF-8)

```
Damlaya damlaya göl olur.
Ağaç yaş iken eğilir.

Bugünün işini yarına bırakma.
Practice makes perfect.
```

File to analyse: proverbs.txt

```
Lines     : 5
Words     : 15
Characters: 98
Letters   : 83
```

CountWords · pieces between spaces, empty pieces ignored

```
static int CountWords(string line)
{
    return line.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
}
```

Exceptions: crash — or catch

without try/catch: File.ReadAllText("grades.txt")

```
Unhandled exception.
System.IO.FileNotFoundException:
Could not find file '…/MyProject/grades.txt'.
```

without try/catch: int.Parse("7O")

```
Unhandled exception. System.FormatException:
The input string '7O' was not in a correct format.
```

the try / catch pattern

```
try
{
    string text = File.ReadAllText("grades.txt");
    Console.WriteLine(text);       // skipped
}
catch (FileNotFoundException)
{
    Console.WriteLine("grades.txt is missing.");
}
Console.WriteLine("The program goes on.");
```

Example 7 with a typing error in the file name: caught, no crash

```
File to analyse: proverb.txt
Sorry, there is no file called "proverb.txt" in this folder.
```

>_ EXAMPLE 11 · CATCH ORDER

Several catch blocks: the specific one first

14_11_SafeReading · part 2

```
string oldPath = Path.Combine("old_data", "scores.txt");
try
{
    string text = File.ReadAllText(oldPath);
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.WriteLine("2) the file is missing");
}
catch (IOException)
{
    Console.WriteLine("2) IOException: no folder old_data");
}
```

Console

```
2) IOException: no folder old_data
```

dotnet build · IOException caught first

```
error CS0160: A previous catch clause already
catches all exceptions of this or of a super
type ('IOException')
```

Exception	Typical cause
`FileNotFoundException`	file missing (typo, wrong folder)
`IOException`	general: no folder, file locked
`FormatException`	int.Parse("abc")

One bad line should not stop the whole file

part 3 · try/catch inside the loop

```
for (int i = 0; i < lines.Length; i++)
{
    try
    {
        sum += int.Parse(lines[i]);
        good++;
    }
    catch (FormatException)
    {
        Console.WriteLine($"3) line {i + 1}: \"{lines[i]}\" skipped");
    }
}
```

part 4 · TryParse: no exception

```
if (!int.TryParse(line, out int value))
{
    bad++;
}
```

scores.txt (made by the program)

```
78
91
abc
65

70
100
```

Console

```
3) line 3: "abc" skipped
3) line 5: "" skipped
3) line 6: "70" skipped
   4 valid scores, average 83.50
4) TryParse: 3 bad lines
```

The record of the guessing game survives

```
inside LoadHighScore: (string name, int attempts)
try
{
    string[] parts = File.ReadAllText(path).Split(',');
    if (parts.Length != 2)
    {
        return ("", 0);
    }
    return (parts[0], int.Parse(parts[1]));
}
catch (FormatException)
{
    Console.WriteLine("(damaged high-score file ignored)");
    return ("", 0);
}
```

run 2

```
Record to beat: 7 attempts by Ayla
Your name: Deniz
…
Correct, Deniz! 6 attempts.
New record saved!
```

run 3

```
Record to beat: 6 attempts by Deniz
Your name: Mert
…
Correct, Mert! 7 attempts.
The record stays with Deniz (6).
```

highscore.txt (after run 3)

```
Deniz,6
```

>_ EXAMPLE 9 · COPY AND FILTER

Read one file, write only the matching lines

14_09_FilterCopy · two streams at once (total and copied are int counters)

```
using (StreamReader reader = new StreamReader(source))
using (StreamWriter writer = new StreamWriter(target))
{
    string line;
    while ((line = reader.ReadLine()) != null)          // read, then test for the end
    {
        total++;
        if (line.Contains(keyword, StringComparison.OrdinalIgnoreCase))
        {
            writer.WriteLine(line);
            copied++;
        }
    }
}
```

Keyword: exam → 2 of 5 lines copied

```
08.10 Final exam: date will be announced
12.10 Exam rooms: see the course group
```

Keyword: lab → 2 of 5 lines copied

```
05.10 Lab 13 solutions are online
09.10 Lab 14 is about text files
```

The balance is saved after every transaction

load: missing → 0, damaged → warning

```
static decimal LoadBalance(string path)
{
    if (!File.Exists(path))
    {
        return 0;
    }
    try
    {
        return decimal.Parse(File.ReadAllText(path));
    }
    catch (FormatException)
    {
        Console.WriteLine($"Warning: {path} damaged; using 0");
        return 0;
    }
}
```

run 2 · loaded; option 3 = history.csv

```
Balance loaded: 1300.00 TL
…
2026-10-12 21:29,deposit,1500.00,1500.00
2026-10-12 21:29,withdraw,200.00,1300.00
2026-10-12 21:29,deposit,250.50,1550.50
```

run 3 · balance.txt contains 1.550,50

```
Warning: balance.txt damaged; using 0
Balance loaded: 0.00 TL
```



Money: decimal, saved with F2 — the program must read what it writes.

>_ COMMON MISTAKES

File bugs we see every year

Mistake	What happens	Fix
`@"C:\Users\ali\data.txt"`	works only on one PC	relative path, Path.Combine
file in another folder	`FileNotFoundException`	dotnet run in the project folder
WriteAllText instead of Append	old content lost	AppendAllText / StreamWriter(path, true)
no using around a StreamWriter	file empty or incomplete	using (…) { … }
ReadLine() twice per pass	every 2nd line skipped	read once, store in line
parsing the header row	`FormatException`	start the loop at i = 1
catch (IOException) first	`CS0160`	specific exception first
empty catch { }	errors are hidden	print a clear message



PART 2

Semester review

One slide per topic: algorithms → flowcharts → variables → operators → decisions → loops → arrays → matrices → methods → files

Algorithm: a finite, exact list of steps

Finite
stops after a finite number of steps

Definite
every step is exact, no guessing

Input / Output
zero or more inputs, at least one output

Effective
each step can really be done

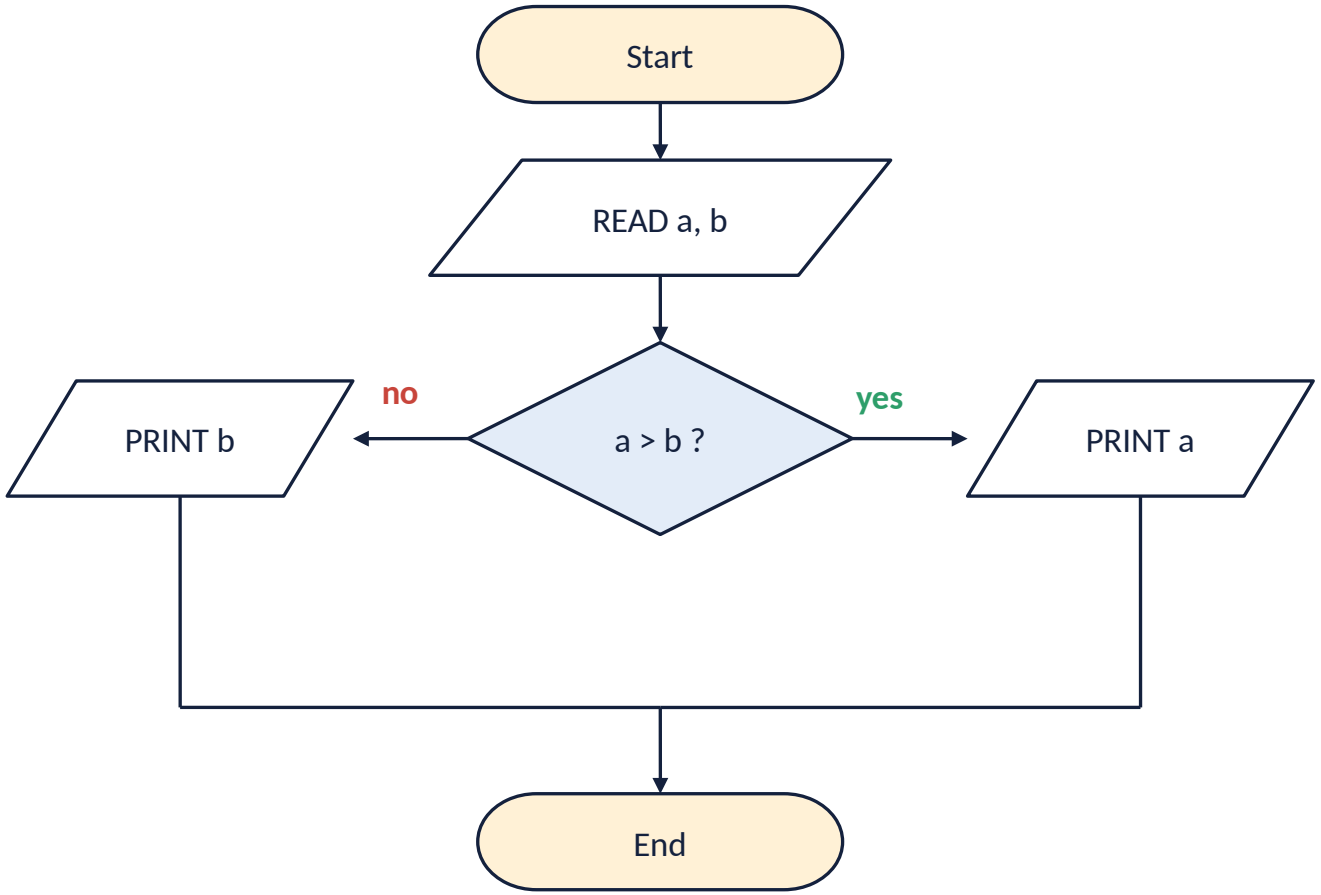
Euclid GCD (Week 1)

```
START
  READ a, b
  WHILE b ≠ 0
    r ← a mod b
    a ← b
    b ← r
  END WHILE
  PRINT a
END
```

a	b	r
48	18	12
18	12	6
12	6	0
6	0	—

Trace GCD(48, 18) → 6. A trace table is the safest way to answer "what does it print?" questions.

Flowcharts: the algorithm as a picture



Symbol	Meaning
rounded (terminal)	Start / End
parallelogram	input / output: READ, PRINT
rectangle	process: x ← x + 1
diamond	decision: two exits yes / no
arrow	flow of control

Rules: one Start, one End; every decision has two labelled exits; every path reaches End.

Variables and data types

declare + initialise

```
int age = 19;
double gpa = 3.25;
decimal price = 49.90m;
char letter = 'A';
string name = "Ada";
bool passed = true;
const double Pi = 3.14159;
```

input and casts

```
int n = int.Parse(Console.ReadLine());
int t = (int)3.9;       // 3: truncation
```

Type	Size	Use for
`int`	4 B	counts, scores
`long`	8 B	big whole numbers
`double`	8 B	measurements
`decimal`	16 B	money (m suffix)
`char`	2 B	one character
`string`	—	text
`bool`	1 B	true / false



ReadLine() returns a string. Always convert: Parse or TryParse.

Operators: integer division, %, precedence

Expression	Value
`7 / 2`	**`3`**
`7 / 2.0`	**`3.5`**
`7 % 3`	**`1`**
`-7 % 3`	**`-1`**
`2 + 3 * 4`	**`14`**
`(2 + 3) * 4`	**`20`**
`10 / 3 * 3`	**`9`**

Expression	Value
`"a" + 1 + 2`	**`"a12"`**
`1 + 2 + "a"`	**`"3a"`**
`a = i++  (i = 5)`	**`a = 5`**
`b = ++i  (i = 6)`	**`b = 7`**
`(int)3.9`	**`3`**

Precedence (high → low)
`!  ++  --  (unary)`
`*  /  %`
`+  -`
`<  <=  >  >=`
`==  !=`
`&&`
`||`
`?:`
`=  +=  -=`



Range test: `x >= 0 && x <= 100` — never `0 <= x <= 100`.

Decisions: if / else if and switch

else-if ladder · first true branch wins

```
if (score >= 90)
{
    letter = "AA";
}
else if (score >= 85)
{
    letter = "BA";
}
else if (score >= 80)
{
    letter = "BB";
}
else
{
    letter = "FF";   // shortened
}
```

switch expression (Example 4)

```
static string LetterGrade(double score)
{
    return score switch
    {
        >= 90 => "AA",
        >= 85 => "BA",
        >= 80 => "BB",
        >= 75 => "CB",
        >= 70 => "CC",
        >= 65 => "DC",
        >= 60 => "DD",
        _ => "FF"
    };
}
```



Order matters: test >= 90 before >= 85, or every 95 becomes BA.

Loops: for, while, do-while

for · counter

```
for (int i = 1; i <= 5; i++)
{
    sum += i;
}
```

while · sentinel

```
while (line != null)
{
    count++;
    line = r.ReadLine();
}
```

do-while · menu

```
do
{
    choice = ReadInt(…);
    // …
} while (choice != 0);
```

Loop	Test	Runs at least	Typical use
`for`	before	0 times	count 1..n, arrays
`while`	before	0 times	until sentinel, until null
`do-while`	after	1 time	menus, validation

Arrays: index from 0, search, sort

5	1	4	2	8
0	1	2	3	4

a a.Length = 5, a[4] = 8

after one bubble-sort pass:

1	4	2	5	8

maximum with its index

```
int maxIndex = 0;                    // start with a[0]
for (int i = 1; i < a.Length; i++)
{
    if (a[i] > a[maxIndex])
    {
        maxIndex = i;
    }
}
```

Algorithm	Needs	Steps
linear search	any order	`O(n)`
binary search	sorted array	`O(log n)`
bubble sort	—	`O(n²)`
Array.Sort	(built in)	`fast`

Reference type: `int[] b = a;` — b[0] = 9 also changes a[0]. Copy with a loop or Clone().

Matrices: rows × columns, nested loops

	c=0	c=1	c=2
r=0	1	2	3
r=1	4	5	6

`m[1, 2]` = 6 (row 1, column 2)

`GetLength(0)` = 2 rows, **`GetLength(1)`** = 3 columns

row sums · outer rows, inner columns

```
for (int r = 0; r < m.GetLength(0); r++)
{
    int rowSum = 0;              // reset per row
    for (int c = 0; c < m.GetLength(1); c++)
    {
        rowSum += m[r, c];
    }
    Console.WriteLine($"row {r}: {rowSum}");
}
```

Typical uses: grade books (students × exams), seat plans, game boards, images — and a CSV file is a matrix written as text!

Methods: parameters, return, scope

Example 12 · a pure, value-returning method

```
static double Average(int midterm, int final)
{
    return 0.4 * midterm + 0.6 * final;
}
```

pass by value

```
int n = 5;
Change(n);              // n is still 5

static void Change(int x)
{
    x = 10;              // changes the copy only
}
```

by value / ref

int copied; arrays: the reference is copied; ref: the variable itself

return / void

return ends the method; every path must return (CS0161)

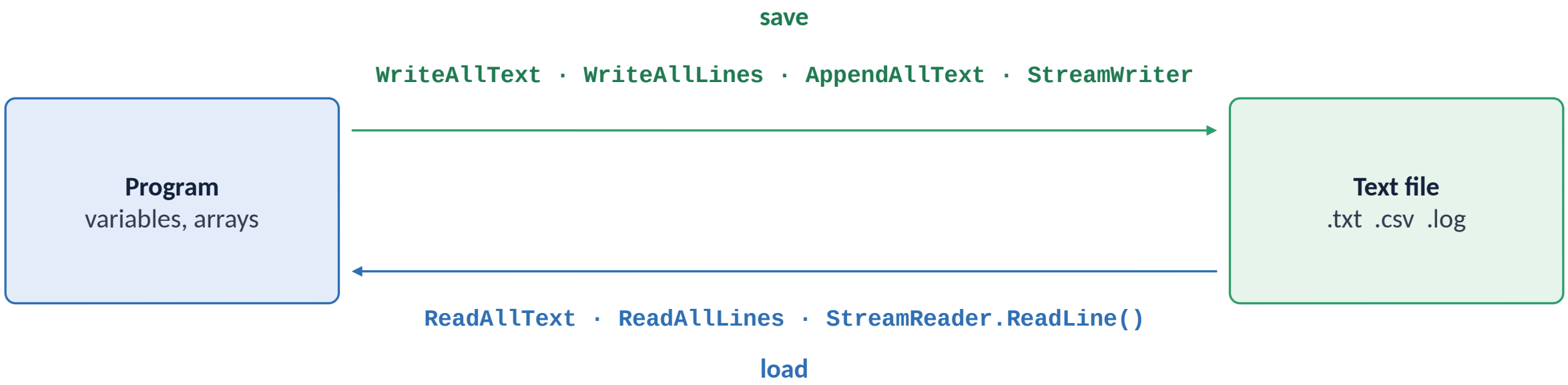
scope

a name lives in its block: CS0103 outside, CS0136 if reused

recursion

base case + smaller step; one stack frame per call

Files: save, append, load



Relative paths
dotnet run → project folder

try / catch
FileNotFound · IO · Format

CSV
Split(',') + int.Parse, skip the header



PART 3

Mini-project: Student Grade Manager

Menu + parallel arrays + methods + CSV save / load — the whole semester in one program

What it does — and how the data is stored

start of the first run

```
=== Student Grade Manager ===
0 student(s) loaded

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 3
```

Menu

add · list · statistics · search · save · exit

Persistence

load at start; save with 5 and on exit

names	"Ayse Kaya"	"Burak Demir"	"Ceren Yildiz"	"Deniz Arslan"	null	…
midterms	72	55	88	58	0	…
finals	64	66	95	61	0	…
	0	1	2	3	4	5

count = 4 · index 1 = Burak Demir in all three arrays · Capacity = 30 places (index 0 … 29)

Ten small methods, each from a week you know

Method	Kind	Uses	Week
`ReadInt(prompt, min, max)`	int, input	TryParse loop, range check	7, 12
`ReadName(prompt)`	string, input	Trim, Contains(',')	12
`Average(midterm, final)`	double, pure	0.4 · mid + 0.6 · fin	4, 13
`LetterGrade(score)`	string, pure	switch expression	5
`IndexOfName(names, count, name)`	int, pure	linear search, −1	10
`AddStudent(…, count)`	int: new count	arrays as parameters	9, 12
`PrintList(…)`	void	for loop, alignment {x,4}	6, 9
`PrintStatistics(…)`	void	sum, max with index, early return	10, 13
`SaveToFile(path, …)`	void	StreamWriter in using	14
`LoadFromFile(path, …)`	int: count	ReadAllLines, Split, try/catch	14

Declare the arrays, then load what was saved

14_12_GradeManager · lines 4–13

```
const int Capacity = 30;
string[] names = new string[Capacity];
int[] midterms = new int[Capacity];
int[] finals = new int[Capacity];
string dataFile = "students.csv";

Console.WriteLine("=== Student Grade Manager ===");
int count = LoadFromFile(dataFile, names, midterms, finals);
Console.WriteLine($"{count} student(s) loaded");
```

run 1 · no file yet

```
=== Student Grade Manager ===
0 student(s) loaded
```

run 2 · loaded from students.csv

```
=== Student Grade Manager ===
4 student(s) loaded
```

const Capacity = 30
one place to change the size; arrays never grow by themselves

count
how many places are used: 0 … 30; every loop runs i < count

A do-while menu with switch — almost pseudocode

lines 16–50 (shortened)

```
do
{
    Console.WriteLine();
    Console.WriteLine("1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit");
    choice = ReadInt("Choice: ", 0, 5);
    switch (choice)
    {
        case 1:
            count = AddStudent(names, midterms, finals, count);
            break;
        case 2:
            PrintList(names, midterms, finals, count);
            break;
        case 3:
            PrintStatistics(names, midterms, finals, count);
            break;
        // case 4: search · case 5: save
    }
} while (choice != 0);
```

count = AddStudent returns the new count. Without "count =" the student is never counted.

No default: ReadInt only returns 0 … 5, so every value has a case (0 ends the loop).

Never trust the keyboard: validate every value

ReadInt · TryParse + range check

```
static int ReadInt(string prompt, int min, int max)
{
    Console.Write(prompt);
    int value;
    while (!int.TryParse(Console.ReadLine(), out value)
           || value < min || value > max)
    {
        Console.Write($"Please type a number from {min} to {max}: ");
    }
    return value;
}
```

run 1 · invalid name and score

```
Name: Deniz, Arslan
Name must not be empty or contain a comma: Deniz Arslan
Midterm (0-100): 58
Final (0-100): 610
Please type a number from 0 to 100: 61
```

run 2 · invalid choice

```
Choice: x
Please type a number from 0 to 5: 3
```

Statistics and search reuse the small methods

PrintStatistics (after the empty check)

```
double sum = 0;
int best = 0, passed = 0;
for (int i = 0; i < count; i++)
{
    double avg = Average(midterms[i], finals[i]);
    sum += avg;
    if (avg > Average(midterms[best], finals[best]))
    {
        best = i;
    }
    if (LetterGrade(avg) != "FF")
    {
        passed++;
    }
}
Console.WriteLine($"Class average: {sum / count:F2}");
Console.WriteLine($"Best student : {names[best]}");
Console.WriteLine($"Passed       : {passed} of {count}");
```

run 2 · statistics

```
Class average: 63.48
Best student : Ceren Yildiz
Passed       : 3 of 5
```

run 2 · search

```
Name to search: ceren yildiz
Ceren Yildiz: average 92.2, grade AA
Name to search: Can
Not found.
```

Reuse: Average and LetterGrade are called by List, Search and Statistics.

SaveToFile: one CSV line per used place

```
SaveToFile
static void SaveToFile(string path, string[] names, int[] midterms, int[] finals, int count)
{
    using (StreamWriter writer = new StreamWriter(path))
    {
        for (int i = 0; i < count; i++)
        {
            writer.WriteLine($"{names[i]},{midterms[i]},{finals[i]}");
        }
    }
    Console.WriteLine($"{count} student(s) saved to {path}");
}
```

students.csv after run 2

```
Ayse Kaya,72,64
Burak Demir,55,66
Ceren Yildiz,88,95
Deniz Arslan,58,61
Emre Sahin,30,41
```

run 2 · exit (automatic save)

```
5 student(s) saved to students.csv
Goodbye!
```

LoadFromFile: split, check, skip damaged lines

LoadFromFile · the loop

```
for (int i = 0; i < lines.Length && count < names.Length; i++)
{
    string[] fields = lines[i].Split(',');
    if (fields.Length == 3
        && int.TryParse(fields[1], out int mid)
        && int.TryParse(fields[2], out int fin))
    {
        names[count] = fields[0];
        midterms[count] = mid;
        finals[count] = fin;
        count++;
    }
    else
    {
        Console.WriteLine($"Line {i + 1} skipped: \"{lines[i]}\"");
    }
}
```

3 fields?
fields.Length == 3, otherwise skip

numbers?
TryParse for both grades

room left?
count < names.Length

Quiz: what do these programs print?

Q1 · write or append?

```
File.WriteAllText("a.txt", "one\n");
File.AppendAllText("a.txt", "two\n");
File.AppendAllText("a.txt", "three\n");
File.WriteAllText("a.txt", "four\n");
File.AppendAllText("a.txt", "five\n");
string[] lines = File.ReadAllLines("a.txt");
Console.WriteLine(lines.Length);
Console.WriteLine(lines[0]);
```

Q2 · CSV

```
string[] rows = { "3,4", "10,2", "7,7" };
File.WriteAllLines("b.csv", rows);
int total = 0;
foreach (string row in File.ReadAllLines("b.csv"))
{
    string[] f = row.Split(',');
    total += int.Parse(f[0]) * int.Parse(f[1]);
}
Console.WriteLine(total);
```

Q3 · bad data

```
string[] data = { "10", "x", "20" };
File.WriteAllLines("c.txt", data);
int sum = 0;
foreach (string s in File.ReadAllLines("c.txt"))
{
    try
    {
        sum += int.Parse(s);
        Console.Write("ok ");
    }
    catch (FormatException)
    {
        Console.Write("bad ");
    }
}
Console.WriteLine(sum);
```

Answers (checked by running the code)

Q1

```
2
four
```

Q2

```
81
```

Q3

```
ok bad ok 30
```

Q1: WriteAllText starts again with "four"; "five" is appended → 2 lines.

Q2: 3·4 + 10·2 + 7·7 = 12 + 20 + 49 = 81

Q3: "x" jumps to catch: "ok " is skipped for that line; 10 + 20 = 30.



Take-away: an exception skips the REST of the try block; with the try inside the loop, the loop goes on.

>_ IN-CLASS EXERCISES

Your turn (pairs, 25 minutes)

E1 · Squares file ★

Write "n,n²" for n = 1..10 to squares.csv with a StreamWriter, read it back and print the sum of the squares (385).

E2 · Numbered copy ★★

Copy proverbs.txt to numbered.txt as "1: …", "2: …", skipping empty lines. Print how many lines were written.

E3 · Shopping CSV ★★

Create items.csv (item,price,qty), print each line total and the grand total with decimal and F2.

E4 · Find the bug ★

A StreamReader loop calls ReadLine() twice per pass. Predict which lines are printed, then fix it.

>_ HOMEWORK

Before the final review

H1 · Sort the students ★★

Add menu option 6 to the Grade Manager: bubble sort by average (highest first), swapping the three arrays together.

H2 · Word frequency ★★

int CountWord(string path, string word): how often does a word appear in a text file (ignore case)?

H3 · Matrix to CSV and back ★★★

Save an int[3, 4] matrix as CSV (one row per line) and load it back into a new int[,]; print both.

H4 · Diary ★

Append a typed sentence with a timestamp to diary.txt; then print the last three entries.

Submit the Program.cs files on the course group, with one console run each. Do not submit data files.

>_ SUMMARY

Key Takeaways

01 Variables die with the program; files on the disk survive. Relative paths start in the working directory.

02 File.WriteAllText replaces, AppendAllText adds, ReadAllText / ReadAllLines load (arrays!).

03 StreamWriter / StreamReader inside using; ReadLine() until null.

04 CSV: one record per line, Split(','), int.Parse, skip the header.

05 try / catch: FileNotFoundException, IOException, FormatException — specific first.

06 The Grade Manager joins menus, arrays, methods and files: the whole semester.

NEXT WEEK

Week 15: Final exam review



MCMP113
Course materials